

RAG against the machine

Guia de Estudo para a Peer-Evaluation

Este documento explica, módulo a módulo, o que o projeto faz, que ferramentas usa e porquê, e resume a auditoria de conformidade feita ao código (o que estava certo, o que foi corrigido, e o estado final).

Projeto: RAG against the machine (42 School)

Objetivo: sistema RAG que responde a perguntas sobre o código-fonte do vLLM

Modelo de geração: Qwen/Qwen3-0.6B

Método de retrieval: BM25 (rank_bm25)

1. O que é RAG?

RAG (Retrieval-Augmented Generation) combina dois sistemas: **Retrieval** — dada uma pergunta, vai buscar os documentos mais relevantes a uma base de dados — e **Generation** — passa esses documentos a um LLM que gera uma resposta baseada neles.

Em vez de o modelo "saber" tudo de memória (o que exige re-treino caro), o RAG dá-lhe acesso a uma fonte externa de informação em tempo real. As quatro fases descritas no subject são:

- **Indexing** — estrutura os dados para serem pesquisáveis (chunking + BM25).
- **Retrieving** — codifica a query, procura no índice, ordena por relevância.
- **Augmenting** — filtra o ruído e insere os melhores chunks na janela de contexto do LLM.
- **Generating** — o LLM lê o contexto e escreve a resposta final.

Analogia: um exame com consulta. O RAG é o sistema que te ajuda a encontrar a página certa no livro (retrieval) antes de escreveres a resposta (generation).

2. Arquitetura do Projeto

```
src/student/  
■■■ models.py      # Estruturas de dados (Pydantic)  
■■■ chunker.py     # Divide ficheiros em pedaços (AST / parágrafos)  
■■■ indexer.py     # Constrói, guarda e pesquisa o índice BM25  
■■■ generator.py   # Gera respostas com o Qwen3-0.6B  
■■■ __main__.py    # CLI (Python Fire) – liga tudo
```

Pipeline completo:

```
vLLM repo -> chunker -> indexer (BM25) -> [pergunta] -> retriever  
-> generator -> resposta em JSON (Pydantic)
```

3. models.py — Estruturas de Dados

O que é Pydantic e porquê usá-lo?

Pydantic é uma biblioteca que valida dados automaticamente a partir de anotações de tipo Python. Defines a estrutura esperada (uma classe que herda de `BaseModel`) e ele garante, em runtime, que os dados recebidos batem certo — caso contrário, lança uma `ValidationError` clara em vez de deixar um erro silencioso propagar-se pelo pipeline.

- **Validação automática:** campo em falta ou tipo errado dá erro imediato, não um bug escondido três funções à frente.
- **Serialização JSON:** `model.model_dump_json()` produz exatamente o JSON que a moulinette espera.
- **Exigido pelo subject:** "All classes must use pydantic for validation and type safety" (IV.1).

```
class MinimalSource(BaseModel):  
    file_path: str  
    first_character_index: int  
    last_character_index: int
```

Modelos implementados

Modelo	Para que serve
MinimalSource	Uma fonte: ficheiro + índices de início/fim do chunk
UnansweredQuestion	Pergunta sem resposta (input do dataset)
AnsweredQuestion	Pergunta + resposta + fontes (ground truth)
RagDataset	Lista de perguntas (dataset completo)
MinimalSearchResults	question_id + question_str + fontes recuperadas
MinimalAnswer	MinimalSearchResults + resposta gerada pelo LLM
StudentSearchResults	Lista de MinimalSearchResults + k
StudentSearchResultsAndAnswer	Lista de MinimalAnswer + k

Porquê question_str em vez de question?

O subject PDF (secção V.7) descreve o campo como question, mas o **binário real da moulinette** (confirmado ao correr moulinette-ubuntu e ao ler o README do pacote) espera question_str no output de search/answer. Como o binário é quem realmente avalia o projeto, é a fonte da verdade — e foi confirmado nos testes: corremos o binário oficial sobre o output do projeto e ele validou os dados ("Student data is valid: True") e devolveu os mesmos números de Recall@k que o comando evaluate próprio do projeto.

4. chunker.py — Estratégia de Chunking

Um ficheiro pode ter dezenas de milhares de caracteres — indexá-lo como uma unidade única seria demasiado genérico para retrieval. O chunking divide cada ficheiro em pedaços mais pequenos e semanticamente coerentes. **Limite:** máximo 2000 caracteres por chunk (exigido pelo subject V.4 e configurável via --max_chunk_size).

Python (.py) — chunking por AST

```
import ast
tree = ast.parse(content)
# fronteiras de chunk em cada 'def' e 'class' de topo
```

O **AST (Abstract Syntax Tree)** é a representação estrutural do código Python — a árvore que o interpretador constrói antes de executar. Ao dividir nas fronteiras de def e class, cada chunk fica com uma unidade lógica completa (uma função ou uma classe inteira). Isto é essencial para retrieval de código: se alguém pergunta "como funciona a função X?", queremos recuperar o chunk com a função inteira, não metade dela cortada a meio. Se o ficheiro tiver um erro de sintaxe (SyntaxError), há fallback automático para chunking por parágrafos.

Texto / Markdown — chunking por parágrafos

```
paragraphs = content.split('\n\n')
# parágrafos adjacentes são fundidos até ao limite de 2000 chars
```

A documentação Markdown do vLLM é naturalmente dividida por linhas em branco. Cada parágrafo tende a focar-se num tópico — ideal como unidade de retrieval. Parágrafos grandes a mais são ainda divididos por linha (_split_by_size).

Garantia de tamanho máximo (3 camadas)

- 1) Tratamento específico de blocos/parágrafos grandes durante o chunking inicial.
- 2) `_split_by_size` — divide por linhas quando um bloco excede o limite.
- 3) `_enforce_max_size` — rede de segurança final: qualquer chunk que ainda exceda 2000 chars é cortado à força por caracteres.

Esta redundância existe porque a moulinette rejeita qualquer chunk acima de 2000 caracteres (marcaria os dados como inválidos).

5. indexer.py — Retrieval com BM25

O que é BM25?

BM25 (Best Match 25) é um algoritmo de ranking de documentos baseado em frequência de termos — a evolução estatística do TF-IDF.

- **TF (Term Frequency):** quantas vezes um termo aparece no documento.
- **IDF (Inverse Document Frequency):** termos raros no corpus pesam mais.
- **Term saturation:** a partir de certo ponto, repetir uma palavra deixa de aumentar o score (evita que um ficheiro "ganhe" só por repetir uma keyword).
- **Document-length normalisation:** documentos longos não têm vantagem injusta só por serem maiores.

Porquê BM25 e não embeddings semânticos?

Critério	BM25	Embeddings semânticos
Velocidade	Instantâneo (CPU)	Precisa de um encoder neural por query
Hardware	Não precisa de GPU	Beneficia muito de GPU
Código-fonte	Ótimo p/ nomes exatos (funções, classes)	Pode perder termos exatos
Exigência do subject	"TF-IDF or BM25" — cumprido	Só seria bônus

Tokenização personalizada

```
def tokenize(text):
    tokens = re.split(r'[\^\w_]+', text.lower())
    # + separa camelCase e snake_case em sub-palavras
```

Identificadores em código usam `camelCase` e `snake_case`. Um tokenizador ingénuo trataria `OpenAICompatibleServer` como um único token opaco. A tokenização aqui indexa *tanto* o token completo *como* as suas sub-partes (`openai`, `compatible`, `server...`), para que uma query em linguagem natural como "openai compatible server" encontre o identificador de código equivalente.

Persistência do índice

O `BM25Okapi` não é diretamente serializável de forma limpa junto com metadados legíveis, por isso o índice é guardado em três ficheiros complementares:

- `bm25.pkl` — o modelo BM25 (pickle).
- `chunks_meta.json` — metadados legíveis (`file_path`, `start`, `end`) de cada chunk.
- `chunk_texts.pkl` — o texto de cada chunk (pickle, para reconstrução rápida).

Esta separação permite um *cold start* em menos de 1 segundo (sem re-indexar), bem dentro do limite de 60s exigido pelo subject.

6. generator.py — Geração de Respostas

O modelo: Qwen/Qwen3-0.6B

0.6B = 600 milhões de parâmetros — pequeno o suficiente para correr em CPU sem GPU dedicada, mas com capacidade de linguagem natural razoável. É o modelo exigido pelo subject (IV.2), carregado via HuggingFace transformers (AutoModelForCausalLM + AutoTokenizer).

Deteção automática de device

```
if torch.backends.mps.is_available():
    device = 'mps'      # Apple Silicon (GPU)
elif torch.cuda.is_available():
    device = 'cuda'     # NVIDIA GPU
else:
    device = 'cpu'      # fallback universal (float32)
```

Em CPU usa-se torch.float32; em GPU (CUDA/MPS) usa-se float16 para reduzir memória e acelerar a inferência.

Construção do prompt

Answer the question using only the context below. Be concise.

```
Context:
[ficheiro1.py]
...snippet...
---
[ficheiro2.md]
...snippet...

Question: <pergunta>
Answer:
```

- "using only the context" — reduz alucinação (o modelo inventar factos).
- "Be concise" — respostas curtas e diretas, dentro do orçamento de tokens.
- Nome do ficheiro incluído — a resposta fica "source-grounded", como o subject pede (V.2.3).

Parâmetros de geração

- max_new_tokens — limita o tamanho da resposta (mais rápido, evita divagação).
- do_sample=False — decoding greedy/determinístico: a mesma pergunta dá sempre a mesma resposta (útil para avaliação reprodutível).
- Truncamento do contexto a max_context_length caracteres por fonte, para respeitar a janela de contexto do modelo.

7. __main__.py — CLI com Python Fire

O que é o Fire e porquê?

Python Fire (Google) converte automaticamente qualquer classe ou função Python numa interface de linha de comandos — sem precisar de declarar argumentos manualmente como no `argparse`.

```
class RAGSystem:
    def index(self, ...): ...
    def search(self, ...): ...

fire.Fire(RAGSystem)

# uso:
uv run python -m student index --max_chunk_size 2000
uv run python -m student search "query" --k 10
```

Exigido pelo subject (IV.2): "CLI using Python Fire".

Comandos disponíveis

Comando	O que faz
index	Lê o repo vLLM, chunka os ficheiros, constrói e guarda o índice BM25
search	Pesquisa uma query única e mostra os top-k resultados
search_dataset	Processa um dataset de perguntas, guarda os resultados em JSON
answer	Pesquisa + gera uma resposta com o LLM para uma query única
answer_dataset	Gera respostas com o LLM para um dataset completo de resultados
evaluate	Calcula Recall@k comparando resultados com o ground truth

Avaliação — Recall@k e IoU

Recall@k: para cada pergunta, verifica-se se as fontes corretas (ground truth) estão presentes nos top-k resultados recuperados:

$$\text{Recall@k} = \frac{(\text{nº de fontes corretas encontradas no top-k})}{(\text{total de fontes corretas para essa pergunta})}$$

IoU (Intersection over Union) é usado para decidir se um chunk recuperado "corresponde" a uma fonte do ground truth, comparando os intervalos de caracteres:

$$\text{IoU} = \frac{\text{tamanho_da_intersecção}}{\text{tamanho_da_união}}$$

Exemplo: ground truth = chars 100–200; recuperámos chars 150–300. Intersecção = 150–200 (50 chars); união = 100–300 (200 chars); $\text{IoU} = 50/200 = 25\% \geq 5\% \rightarrow$ fonte considerada encontrada. O limiar de 5% é o exigido pelo subject (VI.1.1) e confirmado no README da moulinette.

8. Ferramentas e Bibliotecas — Resumo

Ferramenta	Para que serve	Porquê
uv	Gestor de pacotes/ambientes	Exigido pelo subject; mais rápido que pip puro
pydantic	Validação/serialização de dados	Exigido: "all classes must use pydantic"
rank_bm25	Implementação do algoritmo BM25	Retrieval eficiente sem GPU
transformers	Carregar/correr o Qwen3-0.6B	Biblioteca padrão HuggingFace p/ LLMs
torch	Framework de deep learning	Motor de tensores usado pelo transformers
accelerate	Gestão de device/dtype no transformers	Necessário nalgumas versões do transformers
fire	Geração automática de CLI	Exigido pelo subject
tqdm	Barras de progresso	Exigido para operações longas (indexação, batch)
flake8	Linting / estilo de código (PEP 8)	Exigido pelo subject; make lint
mypy	Verificação estática de tipos	Exigido: todas as funções devem passar sem erros
ast (stdlib)	Parsing da árvore sintática do Python	Chunking estrutural de ficheiros .py

9. Resultados Finais (validados nesta auditoria)

Todos os números abaixo foram re-confirmados a correr o pipeline completo (index → search_dataset → evaluate) e, adicionalmente, verificados com o **binário oficial** moulinette-ubuntu fornecido em moulinette_pkg/ — os dois deram exatamente os mesmos valores.

Métrica	Resultado	Alvo	Estado
Recall@5 — Docs	0.830	≥ 0.80	PASS
Recall@5 — Code	0.590	≥ 0.50	PASS
Moulinette oficial — Docs	PASS (0.830)	PASS	PASS
Moulinette oficial — Code	PASS (0.590)	PASS	PASS
Tempo de indexação	~10–13 s (16 470 chunks)	≤ 5 min	PASS
flake8 (src/student)	0 erros	0 erros	PASS
mypy --strict (deps reais instaladas)	0 erros	0 erros	PASS

10. Auditoria Desta Sessão — O Que Foi Corrigido

Esta secção documenta os problemas reais encontrados ao executar o projeto (não apenas ler o código) e as correções aplicadas. É útil durante a peer-evaluation para explicares que testaste e melhoraste o projeto de forma crítica.

make lint / make lint-strict falhavam

Erro de mypy em models.py: o tipo de StudentSearchResultsAndAnswer.search_results (List[MinimalAnswer]) era incompatível com o tipo herdado de StudentSearchResults (List[MinimalSearchResults]), por invariância de List em Python. Como esta estrutura é exatamente a exigida pelo subject (V.7), a correção foi anotar a linha com # type: ignore[assignment], documentando o

motivo.

Crashes não tratados em vários comandos

`search_dataset`, `answer`, `answer_dataset` e `evaluate` rebentavam com `tracebacks` completos quando recebiam um ficheiro inexistente, JSON inválido, ou um dataset com o schema errado — exatamente o tipo de input degenerado que o subject diz explicitamente que vai ser testado (V.2.5). Isto viola a regra "if your program crashes due to unhandled exceptions during the review, it will be considered non-functional". Foram criados três helpers — `_load_json_model()`, `_validate_k()` e `_write_json()` — que capturam `FileNotFoundError`, `JSONDecodeError` e `ValidationError` do Pydantic, e terminam com uma mensagem clara em `stderr` e `sys.exit(1)`, sem `traceback`.

k negativo ou zero causava um bug silencioso

Com `k=-5`, o slicing `top_k = sorted(...)[-5]` não dava erro — devolvia milhares de resultados em vez de nenhum, porque o slicing negativo do Python corta do fim da lista completa, não limita o topo. Agora `_validate_k()` rejeita `k <= 0` com uma mensagem clara.

Erro de tipagem só visível com as dependências reais instaladas

Ao instalar mesmo o `torch/transformers` (como a avaliação real vai fazer, em vez de deixar o `mypy` tratar esses imports como `Any`), apareceu um erro genuíno em `generator.py`: `self.model.to(device)` não batia certo com os stubs de tipos do `transformers/accelerate`. É uma incompatibilidade dos stubs da biblioteca, não um bug de runtime — foi silenciada com um `type: ignore[arg-type]` devidamente comentado.

Mensagem enganosa e inconsistência no contexto do LLM

O comando `answer` imprimia "2000 chars max context" mas instanciava `AnswerGenerator()` com o valor por omissão da classe (500 chars) — os dois números não batiam certo. Agora ambos os comandos (`answer` e `answer_dataset`) passam explicitamente `max_context_length=DEFAULT_MAX_CHUNK_SIZE` (2000), alinhado com o limite de chunk do subject.

Lógica morta em _build_prompt (parâmetro repo_path nunca usado)

O código tinha um `if/else` que, tal como escrito, fazia sempre a mesma coisa em ambos os ramos — o parâmetro `repo_path` nunca era realmente usado para resolver caminhos. Foi reescrito para usar `repo_path` como fallback real: se o caminho direto (relativo à diretoria onde o CLI corre) não existir, tenta resolver contra a raiz do repositório indexado. Testado com sucesso nos dois casos.

.gitignore incompleto

A pasta `vllm-0.10.1/` (38 MB) na raiz do projeto — fora de `data/raw/`, que já estava ignorada — não estava coberta pelo `.gitignore`. O subject exige explicitamente (secção IX): "Do not include large data files ... in your repository." Foi adicionada a entrada `/vllm-0.10.1/`.

Confirmado como correto (sem alterações)

- O campo `question_str` (em vez de `question`) — confirmado como correto ao correr o binário oficial da `moulinette`.
- A estratégia de chunking dupla (AST para `.py`, parágrafos para texto/Markdown).
- O uso de BM25 com tokenização `camelCase/snake_case`.
- A estrutura de todos os modelos Pydantic.
- Os scores de `Recall@5` (Docs 0.830, Code 0.590) — validados com o binário oficial.

11. Perguntas Típicas da Peer-Evaluation

P: Porque usaste BM25 em vez de embeddings semânticos?

R: BM25 é instantâneo, corre em CPU sem hardware especial, e para código-fonte — onde nomes exatos de funções e classes importam mais do que semântica difusa — funciona muito bem. Embeddings exigiriam um encoder neural por query e, tipicamente, GPU para ser rápido. O subject pede "TF-IDF or BM25"; escolhemos BM25 por ser a evolução superior do TF-IDF (term saturation + normalização por tamanho de documento).

P: O que é chunking e porque tens duas estratégias?

R: Chunking divide ficheiros grandes em pedaços pesquisáveis. Para Python uso o módulo ast para dividir em def/class de topo, preservando cada função ou classe inteira como unidade. Para Markdown/texto uso parágrafos, porque é a unidade natural em que a documentação já está organizada.

P: Como funciona o Recall@k?

R: Para cada pergunta, verifico se as fontes corretas (do ground truth) aparecem nos top-k resultados que recuperei. Uma fonte conta como "encontrada" se o IoU (interseção sobre união dos intervalos de caracteres) com alguma fonte recuperada for $\geq 5\%$. O Recall final é a média, por pergunta, de encontradas/total_de_fontes_corretas.

P: O que é o IoU e porque não um overlap simples?

R: IoU é o standard para comparar intervalos: interseção a dividir pela união. Um "overlap simples" (ex: só a interseção) favoreceria recuperar um chunk enorme que contivesse a fonte inteira lá dentro, mesmo sem precisão nenhuma. O IoU penaliza isso, porque a união (denominador) cresce com o tamanho do chunk recuperado.

P: Porque question_str em vez de question, se o PDF diz question?

R: O subject PDF (V.7) usa question, mas o binário real da moulinette — confirmado ao correr moulinette-ubuntu e ao ler o README do pacote — espera question_str no formato de output do search/answer. Como é o binário que efetivamente avalia o projeto, tratámo-lo como a fonte da verdade em caso de divergência com o PDF.

P: Porque o file_path inclui data/raw/vllm-0.10.1/... ?

R: O ground truth guarda os paths relativos à raiz do projeto (ex: data/raw/vllm-0.10.1/docs/...). Ao indexar, usamos os.path.relpath(fpath) (relativo à diretoria de trabalho onde o comando index corre — normalmente a raiz do projeto) para gerar paths no mesmo formato, sem hardcoding.

P: Como garantes que o programa não rebenta com inputs inválidos?

R: Todos os comandos que leem ficheiros de dataset ou resultados passam por um helper único (_load_json_model) que apanha FileNotFoundError, JSONDecodeError e ValidationError do Pydantic e termina com uma mensagem clara em vez de um traceback. O valor de k é validado (tem de ser positivo) antes de ser usado em qualquer slicing.

P: Porque usaste type: ignore nalguns sítios? Isso não é fazer batota ao mypy?

R: Não — cada type: ignore está anotado com o código de erro específico (ex: [assignment], [arg-type]) e um comentário a explicar a causa exata. Um é uma limitação conhecida da invariância de List em Python quando se faz override de um campo com um subtipo (o próprio subject define esta hierarquia de modelos assim). O outro é uma incompatibilidade dos stubs de tipos do transformers/accelerate com uma chamada .to(device) que funciona perfeitamente em runtime. Nenhum dos dois esconde um erro de lógica

real — foram verificados a correr o código.

12. Checklist Final de Conformidade

Requisito do subject	Estado
Python 3.10+	OK (testado em 3.12)
flake8 sem erros	OK — confirmado
mypy sem erros (com deps reais)	OK — corrigido e confirmado
Todas as classes usam Pydantic	OK
Exceções tratadas graciosamente	Corrigido nesta sessão
Recursos (ficheiros) geridos com context managers	OK
Type hints em funções/parâmetros/retornos	OK
Docstrings PEP 257	OK
Makefile: install/run/debug/clean/lint/lint-strict	OK
Chunking Python (AST) + texto (parágrafos)	OK
Limite de 2000 chars, configurável via CLI	OK
BM25 ou TF-IDF implementado	OK (BM25)
CLI Fire: index/search/search_dataset/answer/ answer_dataset/evaluate	OK
Barras de progresso (tqdm) em operações longas	OK
Qwen/Qwen3-0.6B usado para geração	OK (código validado; pesos não descarregáveis neste sandbox)
Recall@5 ≥ 0.80 (docs) e ≥ 0.50 (code)	OK — 0.830 / 0.590
Indexação ≤ 5 min	OK — ~13s
README com todas as secções exigidas	OK
Secção de uso de IA no README	OK
.gitignore sem dados/pesos grandes	Corrigido nesta sessão

Nota: não foi possível descarregar os pesos reais do Qwen/Qwen3-0.6B neste ambiente de auditoria (sem acesso a huggingface.co). O código de geração (generator.py) foi testado ponta-a-ponta com um modelo GPT-2 minúsculo local, confirmando que o carregamento, a construção do prompt, a leitura dos ficheiros de origem e a geração/decodificação funcionam corretamente. Recomenda-se correr uma vez `make answer_dataset` num ambiente com internet antes da entrega, só para confirmar a descarga e geração reais.